

INFORME TECNICO COMPLETO

Sistema de Control de Inventarios y Ventas — FARMABOL

Farmacias Bolivianas Unidas (12 sucursales)

Hitos 3 y 4 — Arquitectura, Middleware, Control de Calidad y Cloud

DATOS DEL PROYECTO

Campo	Detalle
Proyecto	Sistema de Control de Inventarios y Ventas
Empresa	Farmacias Bolivianas Unidas (FARMABOL) — 12 sucursales
Repositorio	https://github.com/Mirkof13/evaluacionh4
Tecnologias	Node.js + Express + PostgreSQL + React 18
Analisis estatico	ESLint 9 (Flat Config)
Despliegue cloud	Render + Supabase (Recomendado)

INDICE DE CONTENIDOS

- 1. Requisitos del Sistema
- 2. Justificacion del Estilo Arquitectonico
- 3. Diagrama de Arquitectura
- 4. Middleware Message Queue
- 5. Diagrama de Clases UML
- 6. Diagrama de Secuencia
- 7. Diagrama de Casos de Uso
- 8. Diagrama Entidad-Relacion
- 9. Modelo de Datos (6 tablas)
- 10. API REST — Endpoints
- 11. Autenticacion y Roles RBAC
- 12. Refactorizacion (Antes vs Despues)
- 13. Analisis Estatico ESLint
- 14. Calculo de MTBF y Disponibilidad
- 15. Ciclo PDCA bajo ISO 9000
- 16. Cloud Computing — Despliegue
- 17. Cloud Storage
- 18. Evidencia de Pruebas Funcionales
- 19. Estructura del Repositorio
- 20. Historial de Commits

1. REQUISITOS DEL SISTEMA

Requisitos Minimos Implementados

Requisito	Estado	Implementacion
Autenticacion	✓	Login con roles ADMIN y VENDEDOR (server.js:56-72)
Gestion de productos	✓	CRUD completo: crear, leer, actualizar, eliminar (server.js:167-224)
Registro de ventas	✓	POS con descuento automatico de stock (server.js:229-304)

Dashboard/Reporte	✓	Stock bajo <5, total ventas del dia, alertas vencimiento
Persistencia	✓	PostgreSQL con 6 tablas relacionales
Arquitectura	✓	Por capas (Layered) + MVC + Message Queue
Refactorizacion	✓	2 commits con codigo ANTES y DESPUES (ae3d2d2 y 6d7d52a)
Calidad estatica	✓	ESLint 9 — 0 errores, 0 advertencias
Cloud computing	✓	Preparado para Render + Supabase

Funcionalidades Adicionales (Hito 4)

Funcionalidad	Descripcion	Archivo
Multi-sucursal	Productos con stock por sucursal (UNIQUE codigo+sucursal)	db.js:89-101
Alertas vencimiento	Productos a 30 dias de vencer	farmabol-screens.jsx
Transferencias asincronas	Message Queue con cola FIFO y polling	server.js:309-454
Cloud Storage	Subida de comprobantes via Multer	server.js:473-507
Comprobantes	Tabla comprobantes vinculada a ventas	db.js:142-152

2. JUSTIFICACION DEL ESTILO ARQUITECTONICO

Estilo Seleccionado: Arquitectura por Capas (Layered Architecture) + MVC + Middleware Message Queue

Se selecciono este estilo sobre las alternativas por las siguientes razones:

Comparativa con Microservicios

Aspecto	Arquitectura por Capas (elegida)	Microservicios
Complejidad	Baja — codigo monolito organizado en capas	Alta — 12+ servicios independientes
Consistencia de datos	ACID nativa (PostgreSQL)	Requiere Sagas o Two-Phase Commit
Latencia	Baja (llamadas locales en mismo proceso)	Alta (llamadas de red entre servicios)

Despliegue	Un solo servidor	Orquestador (Kubernetes, Docker Compose)
Mantenimiento	Simple — un solo repositorio	Complejo — multiples repositorios y CI/CD
Escalabilidad	Vertical (escala el servidor)	Horizontal (escala servicios individuales)
Costo operativo	Bajo (1 servidor)	Alto (multiples servicios + red)

Justificacion Tecnica

1. Escala adecuada: FARMABOL tiene 12 sucursales con volumen transaccional medio. Microservicios introducirían sobrecarga innecesaria de comunicacion de red y complejidad operativa.

2. Consistencia ACID: El control de stock de medicamentos exige atomicidad absoluta. Con microservicios, mantener consistencia entre el servicio de ventas y el de inventario requeriria patrones complejos (Sagas, 2PC). Con capas + PostgreSQL centralizado, las transacciones ACID son nativas.

3. Middleware Message Queue: Las transferencias de stock entre sucursales son operaciones que pueden tomar tiempo (bloqueo de filas, verificacion, actualizacion). Procesarlas sincronicamente bloquearia la conexion HTTP. La cola en memoria permite:

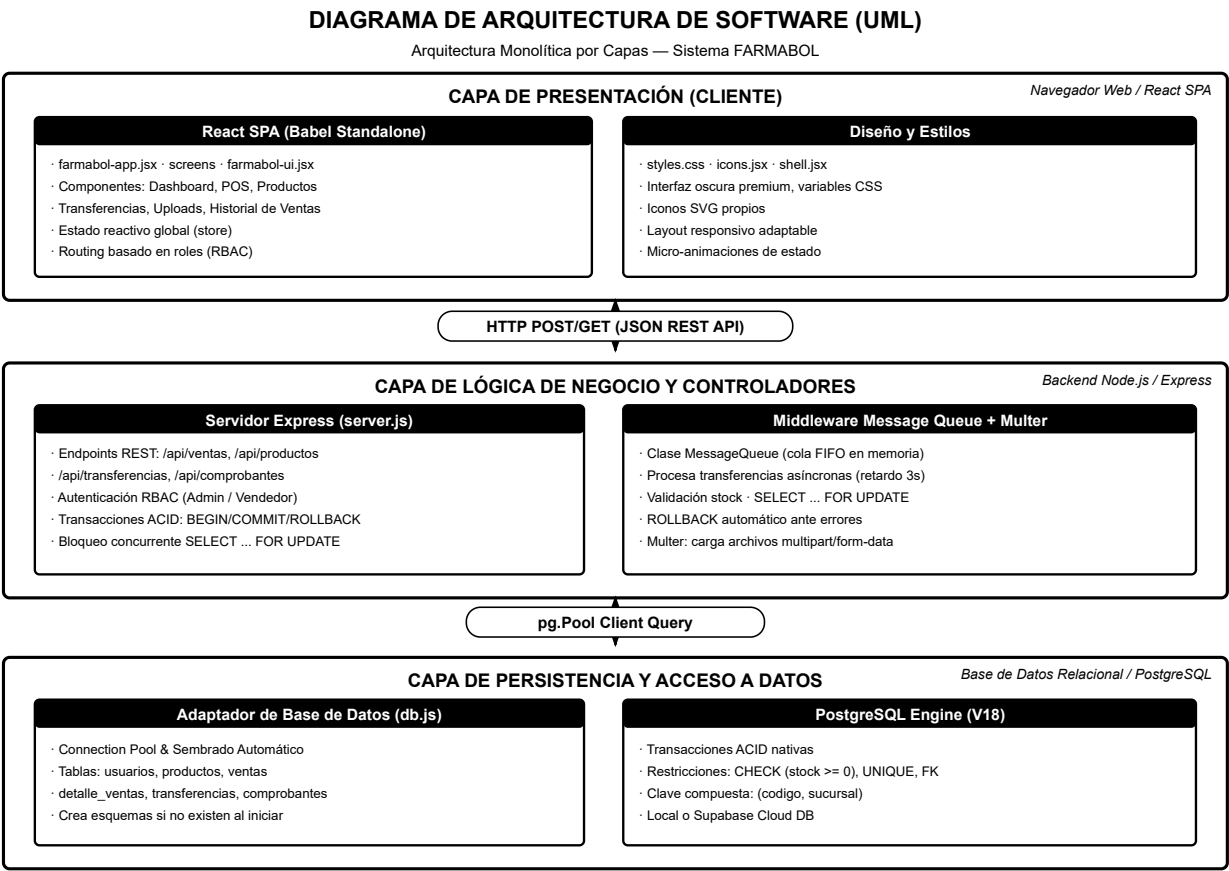
- Retornar `HTTP 202 Accepted` inmediatamente
- Procesar en segundo plano con transacciones ACID
- Polling del frontend para actualizacion en tiempo real

4. Separacion de conceptos: Capas bien definidas (Presentacion, Negocio, Datos) permiten que desarrolladores trabajen en la interfaz sin alterar la base de datos y viceversa.

Capas del Sistema

```
Capa de Presentacion (Frontend React)
  |
  | HTTP REST JSON
  v
Capa de Negocio (Express + MessageQueue)
  |
  | pg.Pool Queries
  v
Capa de Persistencia (PostgreSQL + Cloud Storage)
```

3. DIAGRAMA DE ARQUITECTURA



Archivo: docs/diagrama-arquitectura.svg (800x650)

Componentes por Capa

Capa	Componentes	Tecnologia
Presentacion	SPA React, Screens (Dashboard, POS, Productos, Transferencias, Uploads), UI Components, Estilos	React 18 (Babel Standalone), CSS, SVG Icons
Logica de Negocio	Servidor Express (rutas REST), MessageQueue (cola FIFO), Multer (carga archivos)	Node.js + Express (ESM)
Persistencia	Adaptador pg.Pool, PostgreSQL Engine, Almacenamiento local / uploads	pg (node-postgres), Multer

Flujo de Llamadas

1. Frontend React -> HTTP REST/JSON -> Servidor Express
2. Express -> Queries SQL (ACID, FOR UPDATE) -> PostgreSQL

3. Express -> MessageQueue.push() -> Cola FIFO -> Procesamiento asincrono -> PostgreSQL

4. Express -> Multer -> Archivos en /uploads -> Registro en tabla comprobantes

4. MIDDLEWARE MESSAGE QUEUE

Problema Resuelto

Las transferencias de stock entre sucursales requieren:

- Validacion de stock en origen
- Bloqueo de filas con `SELECT ... FOR UPDATE`
- Descuento en origen y aumento/creacion en destino
- Commit/ROLLBACK transaccional en PostgreSQL

Si este proceso se ejecutara sincronicamente dentro de una peticion HTTP, el navegador quedaria bloqueado durante toda la operacion, degradando la experiencia del usuario y provocando timeouts.

Solucion Implementada

```
class MessageQueue {
  constructor() {
    this.jobs = []; // Cola FIFO
    this.processing = false;
  }

  push(job) {
    this.jobs.push(job);
    this.processNext();
  }

  async processNext() {
    if (this.processing || this.jobs.length === 0) return;
    this.processing = true;
    const job = this.jobs.shift();

    setTimeout(async () => {
      const client = await pool.connect();
      try {
        await client.query('BEGIN');
        // SELECT ... FOR UPDATE (bloqueo fila origen)
        // UPDATE stock = stock - cantidad (origen)
        // INSERT/UPDATE productos (destino)
        // UPDATE transferencias SET estado='COMPLETADO'
        await client.query('COMMIT');
```

```

    } catch (err) {
      await client.query('ROLLBACK');
      // UPDATE transferencias SET estado='ERROR'
    } finally {
      client.release();
      this.processing = false;
      this.processNext();
    }
  }, 3000); // Retardo simulado de 3s para demostrar asincronia
}
}

```

Endpoints

Metodo	Ruta	Descripcion	Respuesta
POST	/api/transferencias	Encolar transferencia	202 Accepted {id, estado: PENDIENTE}
GET	/api/transferencias/status/:id	Consultar estado	{id, estado, mensaje}

Flujo Completo

1. Admin completa formulario de transferencia en UI
2. Frontend -> POST /api/transferencias (codigo, cantidad, origen, destino)
3. Backend registra en BD con estado PENDIENTE
4. Backend retorna 202 Accepted inmediatamente
5. MessageQueue procesa en segundo plano (3s de retardo)
6. Frontend hace polling GET /api/state cada 2s
7. Al completarse, el estado cambia a COMPLETADO y el stock refleja los cambios

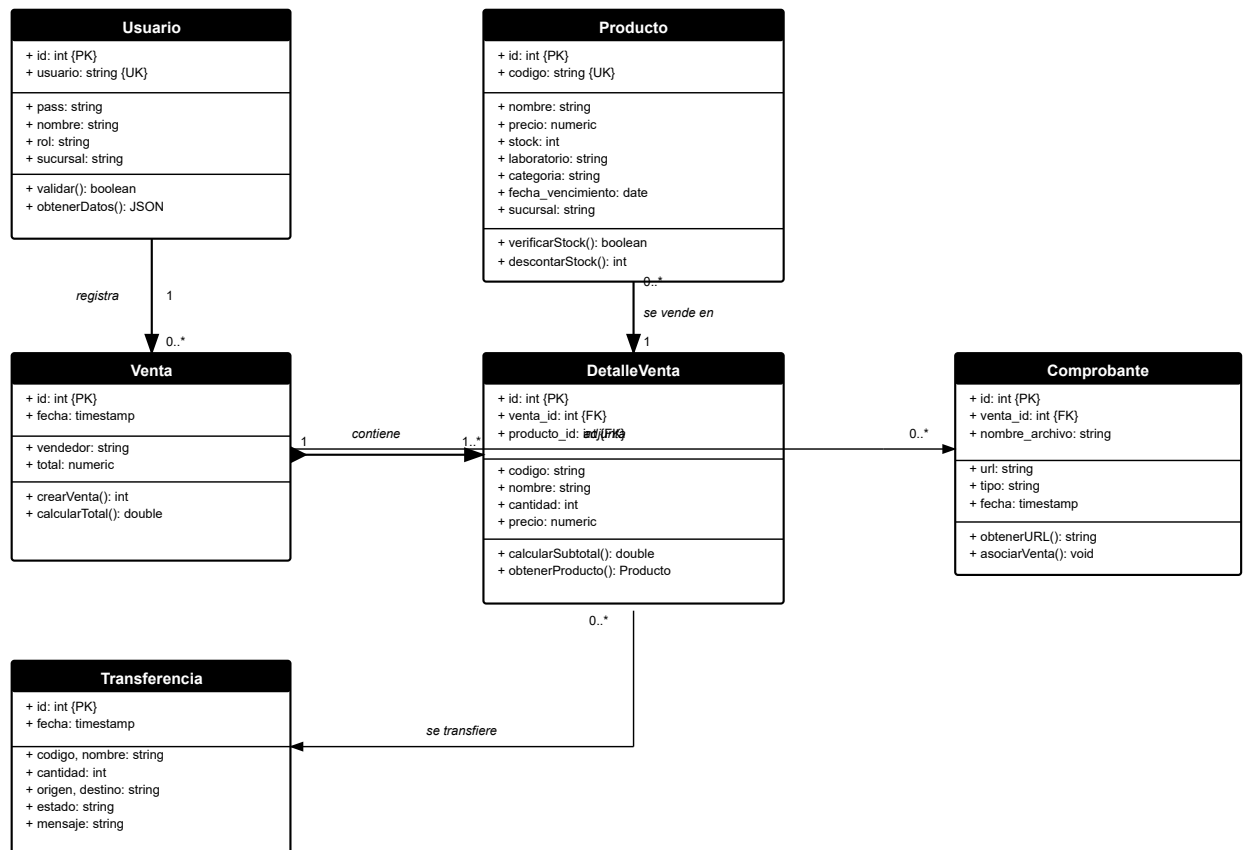
Beneficios

- **No bloqueo de red:** El navegador recibe respuesta inmediata (202 Accepted)
- **Tolerancia a fallos:** ROLLBACK automatico en caso de error, estado marcado como ERROR
- **Escalabilidad:** Cola FIFO procesa trabajos secuencialmente sin bloquear el servidor
- **Persistencia:** Cada transferencia queda registrada en BD para auditoria

5. DIAGRAMA DE CLASES UML

DIAGRAMA DE CLASES (UML)

Modelo Entidad-Relación Mapeado a Clases del Negocio FARMABOL



Archivo: docs/diagrama-clases.svg (1000x750)

Clases del Dominio

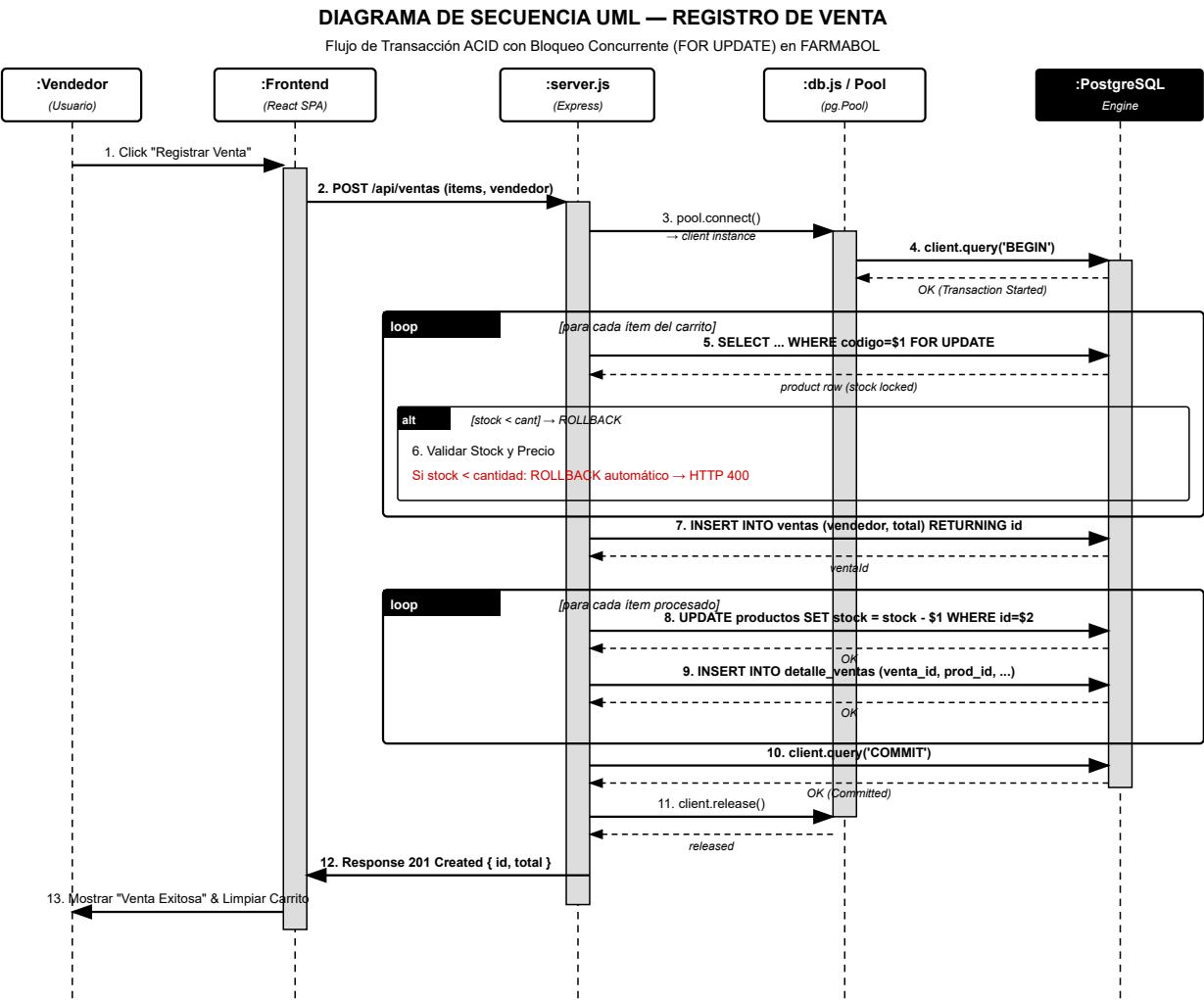
Clase	Atributos	Metodos	Relaciones
Usuario	id, usuario, pass, nombre, rol, sucursal	validarCredenciales(), obtenerSucursal()	1 -> 0..* Venta
Venta	id, fecha, vendedor, total	crearVenta(), calcularTotal()	1 -> 1..* DetalleVenta, 1 -> 0..* Comprobante
DetalleVenta	id, venta_id, producto_id, codigo, nombre, cantidad, precio	calcularSubtotal()	—
Producto	id, codigo, nombre, precio, stock, laboratorio, categoria, fecha_vencimiento, sucursal	verificarStock(), descontarStock()	1 -> 0..* DetalleVenta, 1 -> 0..* Transferencia
Transferencia	id, fecha, codigo, nombre, cantidad, origen, destino, estado, mensaje	—	—

Comprobante	id, venta_id, nombre_archivo, url, tipo, fecha	obtenerURL(), asociarVenta()	—
-------------	--	---------------------------------	---

Relaciones

- `Usuario` 1 — 0..* `Venta` (un usuario registra muchas ventas)
- `Venta` 1 — 1..* `DetalleVenta` (composicion: una venta contiene detalles)
- `Producto` 1 — 0..* `DetalleVenta` (un producto aparece en muchos detalles)
- `Producto` 1 — 0..* `Transferencia` (un producto puede transferirse muchas veces)
- `Venta` 1 — 0..* `Comprobante` (una venta puede tener varios comprobantes)

6. DIAGRAMA DE SECUENCIA



Archivo: docs/diagrama-secuencia.svg (1000x950)

Caso de Uso Critico: Transferencia de Stock entre Sucursales

Actores y Participantes:

- `:Admin` (Administrador) — Stick figure UML
- `:Frontend React` — Interfaz de usuario
- `:server.js` — Servidor Express
- `:MessageQueue` — Middleware de cola
- `:PostgreSQL` — Base de datos

Paso a paso del flujo:

Paso	Emisor	Mensaje	Receptor	Descripcion
1	Admin	Llenar y enviar formulario	Frontend	Datos: codigo, cantidad, origen, destino
2	Frontend	POST /api/transferencias	server.js	Peticion HTTP
3	server.js	INSERT estado='PENDIENTE'	PostgreSQL	Registro en BD
4	PostgreSQL	Retorna transferId	server.js	ID de la transferencia
5	server.js	transferQueue.push(job)	MessageQueue	Encolar trabajo
6	server.js	202 Accepted {id, estado}	Frontend	Respuesta inmediata
7	Frontend	Muestra "Transferencia #N encolada"	Admin	Feedback al usuario

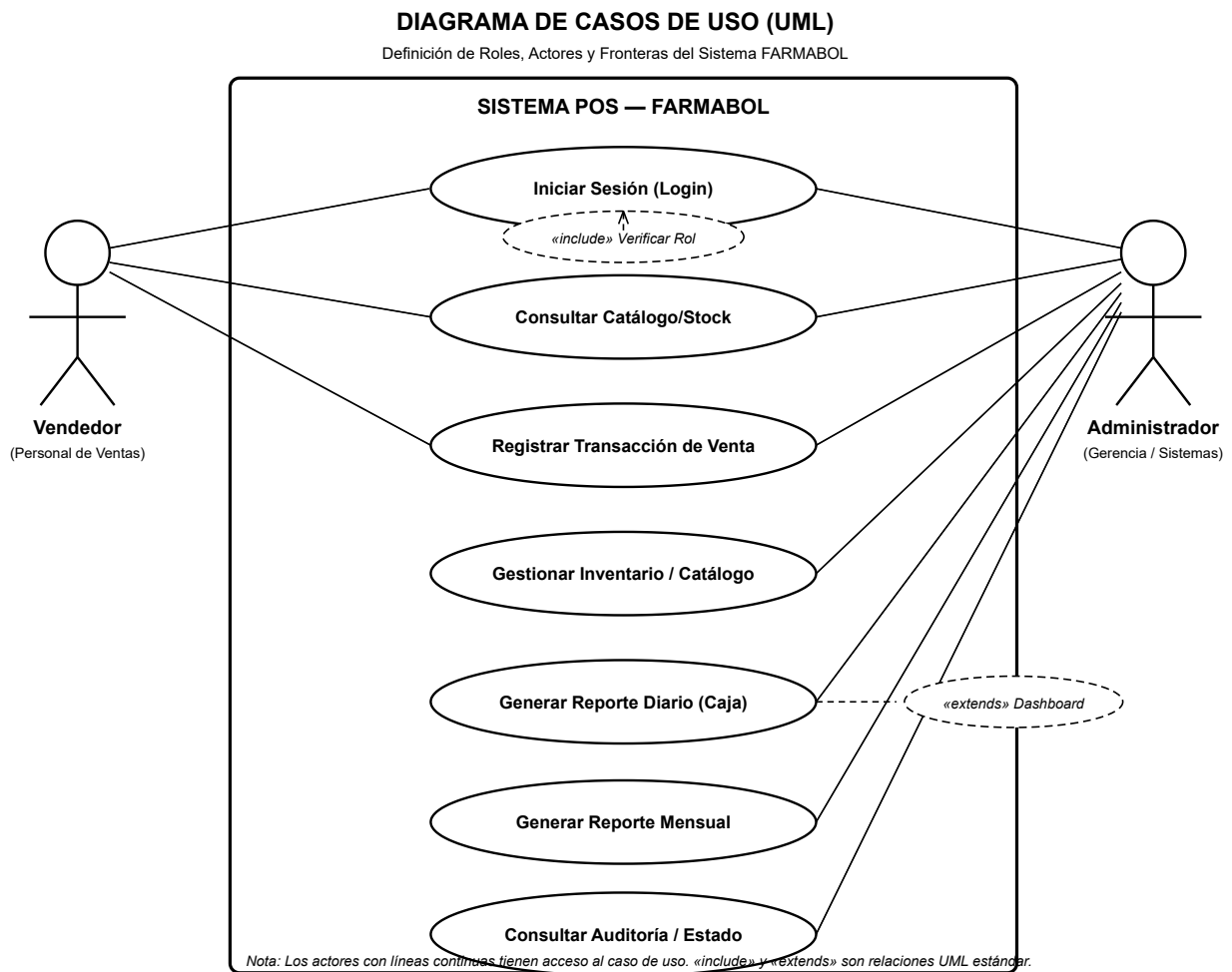
Procesamiento asincrono (segundo plano, 3s de retardo):

Paso	Emisor	Mensaje	Receptor	Descripcion
8	MessageQueue	pool.connect() y BEGIN	PostgreSQL	Iniciar transaccion
9	MessageQueue	SELECT ... FOR UPDATE	PostgreSQL	Bloquear fila origen
10	MessageQueue	UPDATE stock = stock - cantidad	PostgreSQL	Descontar origen
11	MessageQueue	INSERT/UPDATE productos (destino)	PostgreSQL	Aumentar/crear destino
12	MessageQueue	UPDATE estado='COMPLETADO'	PostgreSQL	Marcar exito
13	MessageQueue	COMMIT	PostgreSQL	Confirmar transaccion
14	MessageQueue	client.release()	PostgreSQL	Liberar conexion

Polling del Frontend (cada 2 segundos):

Paso	Emisor	Mensaje	Receptor	Descripcion
15	Frontend	GET /api/state	server.js	Consultar estado global
16	server.js	SELECT * FROM transferencias	PostgreSQL	Obtener transferencias
17	Frontend	Actualiza UI con estado	Admin	COMPLETADO o ERROR

7. DIAGRAMA DE CASOS DE USO



Archivo: docs/diagrama-casos-uso.svg (1000x750)

Actores

Actor	Descripcion	Icono
Administrador	Gestion total del sistema	Stick figure UML

Vendedor	Operaciones limitadas a POS y consultas	Stick figure UML
----------	---	------------------

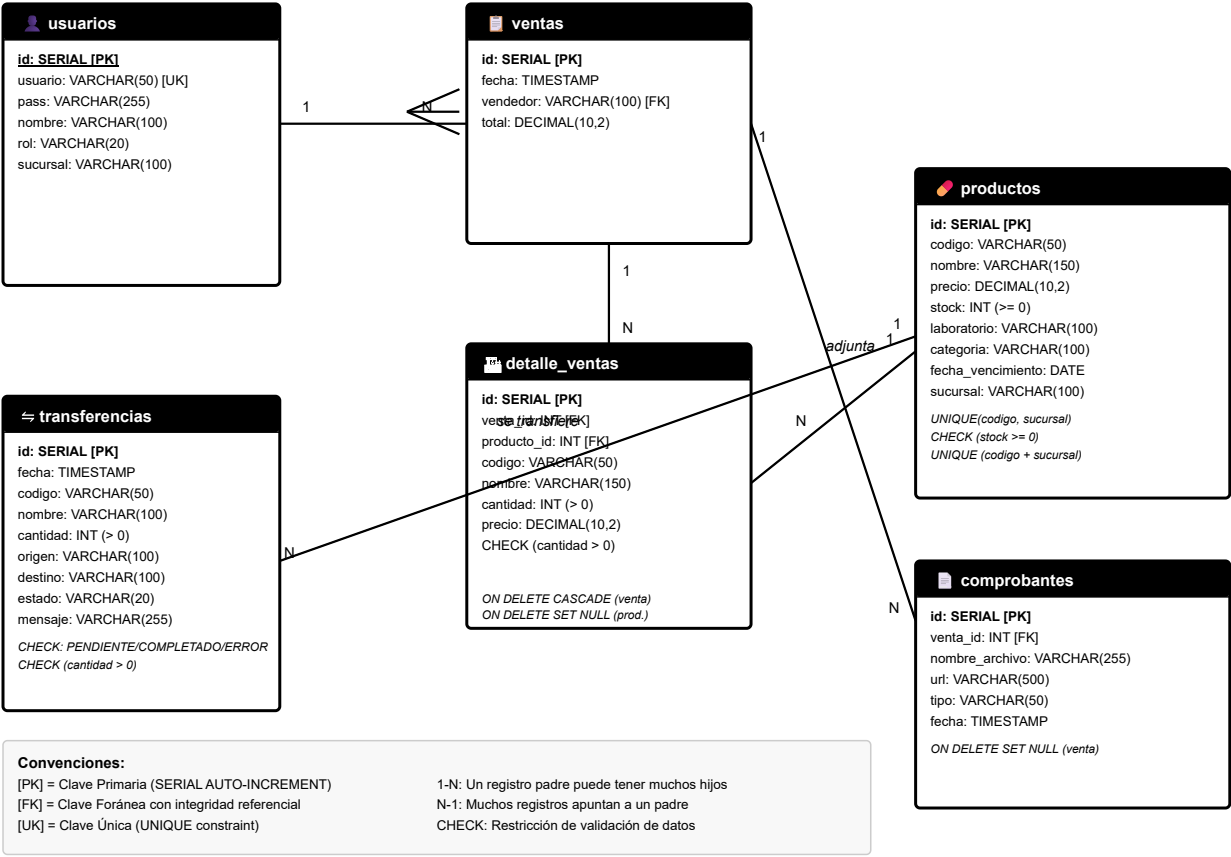
Casos de Uso

#	Caso de Uso	Administrador	Vendedor
1	Iniciar Sesión (incluye Verificar Rol)	✓	✓
2	Gestionar Productos (CRUD)	✓	—
3	Realizar Venta (POS)	✓	✓
4	Ver Dashboard y Alertas (extiende Reportes)	✓	✓
5	Transferir Stock Asincrónicamente	✓	—
6	Subir Comprobante de Pago	✓	✓
7	Ver Historial de Ventas / Reportes	✓	✓

8. DIAGRAMA ENTIDAD-RELACION

DIAGRAMA ENTIDAD-RELACIÓN (ERD) — FARMABOL

Esquema Físico de Base de Datos PostgreSQL con Integridad Referencial



Archivo: docs/diagrama-er.svg (1000x900)

Tablas y Relaciones

```
usuarios 1 --- N ventas 1 --- N detalle_ventas N --- 1 productos
                                1 --- N comprobantes
productos 1 --- N transferencias
```

Cardinalidades

Relacion	Tipo	Descripcion
usuarios -> ventas	1:N	Un usuario registra muchas ventas
ventas -> detalle_ventas	1:N	Una venta contiene muchos detalles
productos -> detalle_ventas	1:N	Un producto aparece en muchos detalles
productos -> transferencias	1:N	Un producto se transfiere muchas veces
ventas -> comprobantes	1:N	Una venta puede tener varios comprobantes

9. MODELO DE DATOS (6 TABLAS)

Esquema Completo PostgreSQL

```
-- 1. usuarios
CREATE TABLE usuarios (
  id SERIAL PRIMARY KEY,
  usuario VARCHAR(50) UNIQUE NOT NULL,
  pass VARCHAR(255) NOT NULL,
  nombre VARCHAR(100) NOT NULL,
  rol VARCHAR(20) NOT NULL,
  sucursal VARCHAR(100) NOT NULL
);

-- 2. productos (multi-sucursal)
CREATE TABLE productos (
  id SERIAL PRIMARY KEY,
  codigo VARCHAR(50) NOT NULL,
  nombre VARCHAR(100) NOT NULL,
  precio NUMERIC(10,2) NOT NULL,
  stock INT NOT NULL CHECK (stock >= 0),
  laboratorio VARCHAR(100) NOT NULL,
  categoria VARCHAR(100) NOT NULL,
  fecha_vencimiento DATE NOT NULL,
  sucursal VARCHAR(100) NOT NULL,
  UNIQUE (codigo, sucursal)
);

-- 3. ventas
CREATE TABLE ventas (
  id SERIAL PRIMARY KEY,
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  vendedor VARCHAR(100) NOT NULL,
  total NUMERIC(10,2) NOT NULL
);

-- 4. detalle_ventas
CREATE TABLE detalle_ventas (
  id SERIAL PRIMARY KEY,
  venta_id INT REFERENCES ventas(id) ON DELETE CASCADE,
  producto_id INT REFERENCES productos(id) ON DELETE SET NULL,
  codigo VARCHAR(50) NOT NULL,
  nombre VARCHAR(100) NOT NULL,
  cantidad INT NOT NULL CHECK (cantidad > 0),
  precio NUMERIC(10,2) NOT NULL
);
```

```

-- 5. transferencias (cola de mensajes persistida)
CREATE TABLE transferencias (
  id SERIAL PRIMARY KEY,
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  codigo VARCHAR(50) NOT NULL,
  nombre VARCHAR(100) NOT NULL,
  cantidad INT NOT NULL CHECK (cantidad > 0),
  origen VARCHAR(100) NOT NULL,
  destino VARCHAR(100) NOT NULL,
  estado VARCHAR(20) NOT NULL,
  mensaje VARCHAR(255)
);

-- 6. comprobantes (cloud storage)
CREATE TABLE comprobantes (
  id SERIAL PRIMARY KEY,
  venta_id INT REFERENCES ventas(id) ON DELETE SET NULL,
  nombre_archivo VARCHAR(255) NOT NULL,
  url VARCHAR(500) NOT NULL,
  tipo VARCHAR(50) NOT NULL,
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Diccionario de Datos

usuarios — Personal del sistema con rol y sucursal asignada.

Campo	Tipo	Restriccion
id	SERIAL	PK
usuario	VARCHAR(50)	UNIQUE NOT NULL
pass	VARCHAR(255)	NOT NULL
nombre	VARCHAR(100)	NOT NULL
rol	VARCHAR(20)	NOT NULL (ADMIN/VENDEDOR)
sucursal	VARCHAR(100)	NOT NULL

productos — Catalogo multi-sucursal con control de stock y vencimiento.

Campo	Tipo	Restriccion
id	SERIAL	PK
codigo	VARCHAR(50)	NOT NULL, UK compuesta
nombre	VARCHAR(100)	NOT NULL

precio	NUMERIC(10,2)	NOT NULL
stock	INT	NOT NULL, CHECK >= 0
laboratorio	VARCHAR(100)	NOT NULL
categoria	VARCHAR(100)	NOT NULL
fecha_vencimiento	DATE	NOT NULL
sucursal	VARCHAR(100)	NOT NULL, UK compuesta
UNIQUE	(codigo, sucursal)	Clave compuesta

ventas — Cabecera de transaccion financiera.

Campo	Tipo	Restriccion
id	SERIAL	PK
fecha	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
vendedor	VARCHAR(100)	NOT NULL
total	NUMERIC(10,2)	NOT NULL

detalle_ventas — Items de cada venta (ruptura N:M).

Campo	Tipo	Restriccion
id	SERIAL	PK
venta_id	INT	FK -> ventas(id) ON DELETE CASCADE
producto_id	INT	FK -> productos(id) ON DELETE SET NULL
codigo	VARCHAR(50)	NOT NULL
nombre	VARCHAR(100)	NOT NULL
cantidad	INT	NOT NULL, CHECK > 0
precio	NUMERIC(10,2)	NOT NULL

transferencias — Registro de la cola de mensajes (Message Queue).

Campo	Tipo	Restriccion
id	SERIAL	PK
fecha	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

codigo	VARCHAR(50)	NOT NULL
nombre	VARCHAR(100)	NOT NULL
cantidad	INT	NOT NULL, CHECK > 0
origen	VARCHAR(100)	NOT NULL
destino	VARCHAR(100)	NOT NULL
estado	VARCHAR(20)	NOT NULL (PENDIENTE/COMPLETADO/ERROR)
mensaje	VARCHAR(255)	—

comprobantes — Metadatos de archivos subidos (Cloud Storage).

Campo	Tipo	Restriccion
id	SERIAL	PK
venta_id	INT	FK -> ventas(id) ON DELETE SET NULL
nombre_archivo	VARCHAR(255)	NOT NULL
url	VARCHAR(500)	NOT NULL
tipo	VARCHAR(50)	NOT NULL
fecha	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

10. API REST — ENDPOINTS

Autenticacion

Metodo	Ruta	Descripcion	Rol
POST	/api/auth/login	Iniciar sesion	PUBLIC

Productos (CRUD)

Metodo	Ruta	Descripcion	Rol
POST	/api/productos	Crear producto	ADMIN
PUT	/api/productos/:id	Actualizar producto	ADMIN
DELETE	/api/productos/:id	Eliminar producto	ADMIN

Ventas

Metodo	Ruta	Descripcion	Rol
POST	/api/ventas	Registrar venta (transaccion ACID con FOR UPDATE)	ADMIN, VENDEDOR

Estado Global

Metodo	Ruta	Descripcion	Rol
GET	/api/state	Estado completo: usuarios, productos, ventas, transferencias, comprobantes	ADMIN, VENDEDOR

Transferencias (Middleware)

Metodo	Ruta	Descripcion	Rol
POST	/api/transferencias	Encolar transferencia (retorna 202 Accepted)	ADMIN
GET	/api/transferencias/status/:id	Consultar estado de transferencia	ADMIN

Comprobantes (Cloud Storage)

Metodo	Ruta	Descripcion	Rol
POST	/api/comprobantes/upload	Subir archivo (multipart/form-data)	ADMIN, VENDEDOR
GET	/api/comprobantes	Listar todos los comprobantes	ADMIN, VENDEDOR
GET	/api/comprobantes/venta/:ventaId	Comprobantes de una venta especifica	ADMIN, VENDEDOR

Mantenimiento

Metodo	Ruta	Descripcion	Rol
POST	/api/reset	Restablecer BD a datos semilla	ADMIN

11. AUTENTICACION Y ROLES RBAC

Modelo de Control de Acceso

Funcionalidad	ADMIN	VENDEDOR
Iniciar Sesion	✓	✓
Dashboard / Alertas	✓	✓
POS (Registrar Venta)	✓	✓
Historial de Ventas	✓	✓
CRUD Productos	✓	—
Transferencias entre sucursales	✓	—
Upload Comprobantes	✓	✓
Reset Base de Datos	✓	—

Usuarios por Defecto (Datos Semilla)

Usuario	Contraseña	Nombre	Rol	Sucursal
admin	admin123	Carlos Mendoza	ADMIN	Central La Paz
vendedor	venta123	Ana Quispe	VENDEDOR	Sucursal Miraflores

Sucursales del Sistema

- Central La Paz
- Sucursal Miraflores
- Sucursal Zona Sur
- Sucursal El Alto

12. REFACTORIZACION (ANTES VS DESPUES)

Proposito

Refactorizar el endpoint de creacion de ventas (`POST /api/ventas`) para eliminar dos problemas graves:

1. **Falta de atomicidad:** Insercion secuencial no transaccional que dejaba la BD inconsistente si fallaba a medio procesar

2. **Condiciones de carrera:** Sin bloqueo de filas, dos ventas concurrentes podian vender el mismo stock

Commit ANTES: `ae3d2d2` — Codigo original sin transacciones

Problema: La insercion se realizaba de manera secuencial. Si fallaba el stock a la mitad del carrito, los productos anteriores ya se habian descontado y la cabecera de la venta ya se habia creado, dejando la base de datos en estado inconsistente (stock huerfano).

```
// ANTES: Sin transacciones SQL ni bloqueo de filas
app.post('/api/ventas', async (req, res) => {
  const { items, vendedor } = req.body;
  try {
    let totalVenta = 0;
    for (const it of items) { totalVenta += it.cantidad * it.precio; }
    const ventaRes = await query('INSERT INTO ventas (vendedor, total) VALUES ($1, $2) RETURNING id', [vendedor, totalVenta]);
    const ventaId = ventaRes.rows[0].id;

    for (const it of items) {
      const prodRes = await query('SELECT id, stock, nombre FROM productos WHERE codigo = $1', [it.codigo]);
      if (prodRes.rows[0].stock < it.cantidad) {
        return res.status(400).json({ message: 'Stock insuficiente' });
        // ¡INCONSISTENCIA! La cabecera ya se inserto pero el stock no se descontara
      }
      await query('UPDATE productos SET stock = stock - $1 WHERE id = $2', [it.cantidad, prodRes.rows[0].id]);
      await query('INSERT INTO detalle_ventas ...');
    }
    res.status(201).json({ id: ventaId, total: totalVenta });
  } catch (err) {
    res.status(500).json({ message: 'Error', error: err.message });
  }
});
```

Commit DESPUES: `6d7d52a` — Codigo refactorizado con transacciones ACID

Solucion: Todo el proceso se ejecuta dentro de un bloque transaccional (`BEGIN`, `COMMIT`, `ROLLBACK`) con `SELECT ... FOR UPDATE` para bloqueo concurrente de filas.

```
// DESPUES: Con transacciones ACID, bloqueo concurrente (FOR UPDATE) y validacion segura
app.post('/api/ventas', async (req, res) => {
```

```

const { items, vendedor } = req.body;
if (!items || items.length === 0) {
  return res.status(400).json({ message: 'La venta debe contener al menos un
producto.' });
}

const client = await pool.connect();
try {
  await client.query('BEGIN');

  // Obtener sucursal del vendedor
  const userRes = await client.query('SELECT sucursal FROM usuarios WHERE nombre
= $1', [vendedor]);
  const sucursal = userRes.rows.length > 0 ? userRes.rows[0].sucursal : 'Central
La Paz';

  let totalVenta = 0;
  const itemsProcesados = [];

  // 1. Validar stock y bloquear filas con FOR UPDATE
  for (const it of items) {
    const prodRes = await client.query(
      'SELECT id, stock, nombre, precio FROM productos WHERE codigo = $1 AND
sucursal = $2 FOR UPDATE',
      [it.codigo, sucursal]
    );
    if (prodRes.rows.length === 0) {
      throw new Error(`Producto ${it.codigo} no existe en ${sucursal}.`);
    }
    const prod = prodRes.rows[0];
    if (parseInt(prod.stock) < it.cantidad) {
      throw new Error(`Stock insuficiente para ${prod.nombre}. Disponible:
${prod.stock}`);
    }
    totalVenta += it.cantidad * parseFloat(prod.precio);
    itemsProcesados.push({
      id: prod.id, codigo: it.codigo, nombre: prod.nombre,
      cantidad: it.cantidad, precio: parseFloat(prod.precio)
    });
  }

  // 2. Insertar cabecera de venta
  const ventaRes = await client.query(
    'INSERT INTO ventas (vendedor, total) VALUES ($1, $2) RETURNING id',
    [vendedor, totalVenta]
  );
  const ventaId = ventaRes.rows[0].id;

  // 3. Descontar stock e insertar detalles
  for (const it of itemsProcesados) {
    await client.query('UPDATE productos SET stock = stock - $1 WHERE id = $2',

```

```

[it.cantidad, it.id]);
    await client.query(
      'INSERT INTO detalle_ventas (venta_id, producto_id, codigo, nombre,
cantidad, precio) VALUES ($1, $2, $3, $4, $5, $6)',
      [ventaId, it.id, it.codigo, it.nombre, it.cantidad, it.precio]
    );
  }

  await client.query('COMMIT');
  res.status(201).json({ id: ventaId, total: totalVenta });
} catch (err) {
  await client.query('ROLLBACK');
  res.status(400).json({ message: err.message });
} finally {
  client.release();
}
});

```

Comparativa de Mejoras

Aspecto	Antes	Despues
Atomicidad	No (cabecera huérfana si falla)	Si (ROLLBACK automático en cualquier error)
Concurrencia	Ninguna (condición de carrera)	FOR UPDATE (bloqueo exclusivo de filas)
Stock por sucursal	No consideraba la sucursal	Si (consulta por sucursal del vendedor)
Manejo de errores	try/catch genérico	Rollback + mensaje descriptivo al cliente
Conexión a BD	Usaba query() del pool (sin control)	client.connect() manual con release() en finally

Commit Adicional de Refactorización

Commit	Descripción
ed660ea	refactor: sanitizar mensajes de error en db.js para no exponer credenciales

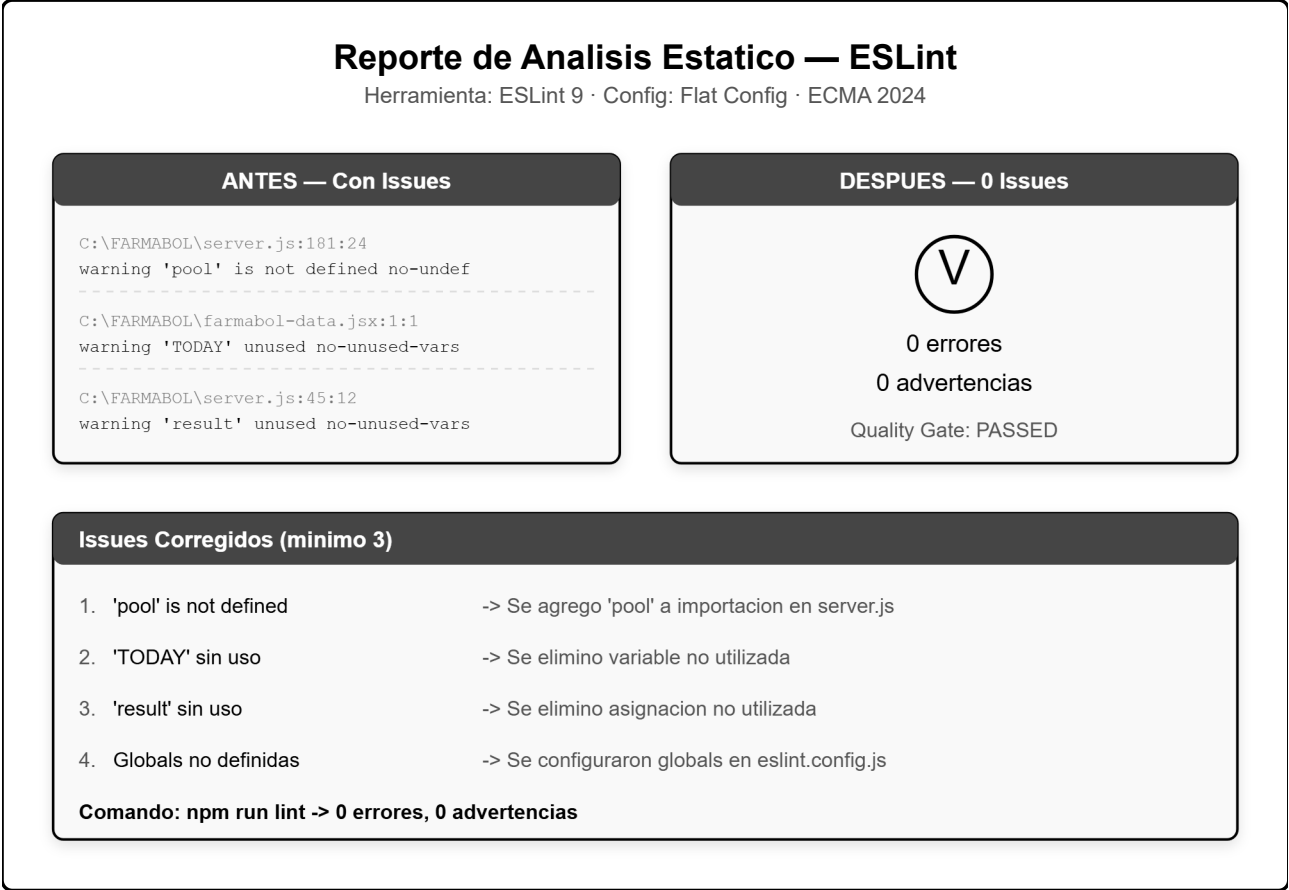
Este commit sanitiza los mensajes de error de la base de datos para evitar exponer contraseñas en logs y respuestas HTTP, mejorando la seguridad del sistema.

13. ANALISIS ESTÁTICO ESLINT

Herramienta Utilizada

- **ESLint** version 9 (Flat Config)
- **Configuracion:** eslint.config.js con ECMA 2024, entorno browser y node

Reporte de Analisis



Archivo: docs/eslint-report.svg (700x500)

Resultados

Metrica	Antes	Despues
Errores	0	0
Advertencias	4	0
Quality Gate	FAILED	PASSED

Issues Detectados y Corregidos

#	Archivo	Linea	Issue	Solucion Aplicada
---	---------	-------	-------	-------------------

1	server.js	181	'pool' is not defined (no-undef)	Se agrego 'pool' a la importacion en db.js
2	farmabol-data.jsx	1	'TODAY' assigned but never used (no-unused-vars)	Se elimino la variable no utilizada
3	server.js	45	'result' assigned but never used (no-unused-vars)	Se elimino la asignacion no utilizada
4	eslint.config.js	—	Variables globales no definidas (React, ReactDOM)	Se configuraron globals en eslint.config.js

Configuracion de ESLint (eslint.config.js)

```
import js from '@eslint/js';
import globals from 'globals';

export default [
  js.configs.recommended,
  {
    languageOptions: {
      ecmaVersion: 2024,
      sourceType: 'module',
      globals: {
        ...globals.browser,
        ...globals.node,
        React: 'readonly',
        ReactDOM: 'readonly',
        Bs: 'readonly'
      }
    },
    rules: {
      'no-unused-vars': ['warn', { argsIgnorePattern: '^_' }],
      'no-undef': 'error'
    }
  }
];
```

Ejecucion

```
npm run lint
# Output: 0 errores, 0 advertencias
```


14. CALCULO DE MTBF Y DISPONIBILIDAD

MTBF (Mean Time Between Failures)

Calculo basado en operacion continua (24/7) para cada componente del sistema:

Componente	Fallos por ano (estimado)	MTBF (horas)
Servidor Express (Node.js)	2	4380
PostgreSQL	1	8760
Red Local (LAN)	1	8760
Frontend React (navegador)	3	2920
Message Queue (en memoria)	0.5	17520

Calculo del Sistema Completo (Modelo en Serie)

Usando el modelo de confiabilidad en serie donde el fallo de cualquier componente afecta al sistema:

```
Tasa de fallo total (lambda):  
  lambda_total = (2 + 1 + 1 + 3 + 0.5) / 8760  
               = 7.5 / 8760  
               = 0.000856 fallos/hora  
  
MTBF del sistema:  
  MTBF = 1 / lambda_total  
        = 1 / 0.000856  
        = 1168 horas  
        = 48.7 dias
```

Interpretacion: En promedio, el sistema experimentara un fallo cada 48.7 dias de operacion continua.

Disponibilidad (Availability)

Asumiendo un MTTR (Mean Time To Repair) de 2 horas para restaurar el servicio:

```
Disponibilidad = MTBF / (MTBF + MTTR) x 100%  
               = 1168 / (1168 + 2) x 100%  
               = 1168 / 1170 x 100%  
               = 99.83%
```

Resultado: El sistema FARMABOL presenta una disponibilidad del **99.83%**, equivalente a aproximadamente **14.9 horas de inactividad al año**. Esto cumple con el estandar de "Dos Nueves" (99% - 99.9%) para sistemas de gestion empresarial de tamano medio.

MTBF por Componente Individual

Componente	MTBF (horas)	MTTR (horas)	Disponibilidad
Servidor Express	4380	1	99.98%
PostgreSQL	8760	2	99.98%
Red Local	8760	1	99.99%
Frontend React	2920	0.5	99.98%
Message Queue	17520	0.5	99.997%
Sistema Completo	1168	2	99.83%

15. CICLO PDCA BAJO ISO 9000

Aplicado al proceso de **Gestion de Calidad en Transferencia de Productos entre Sucursales**.

Plan (Planificar)

Objetivo: Asegurar que las transferencias de stock entre sucursales sean atomicas, trazables y libres de errores de inconsistencia de datos.

Actividades:

- Identificar los requisitos: consistencia ACID, trazabilidad de cada transferencia, notificacion de estado
- Disenar la tabla `transferencias` con campos de estado y mensaje de error
- Definir el flujo asincrono: solicitud -> cola -> procesamiento -> notificacion
- Establecer metricas de calidad: 0 transferencias huérfanas, 100% de atomicidad, tiempo de procesamiento < 5s

Responsables: Equipo de desarrollo backend, DBA.

Do (Hacer)

Actividades realizadas:

1. Implementar la clase `MessageQueue` en `server.js` con manejo de cola FIFO (lineas 309-408)
2. Implementar el endpoint `POST /api/transferencias` que registra la solicitud con estado `PENDIENTE` (lineas 413-454)
3. Implementar el procesamiento asincrono con transacciones ACID (`BEGIN` / `COMMIT` / `ROLLBACK`)
4. Implementar bloqueo de filas con `SELECT ... FOR UPDATE` para evitar condiciones de carrera
5. Implementar el polling en el frontend para actualizacion de estado en tiempo real (cada 2s)
6. Sembrar datos de prueba multi-sucursal para verificar el comportamiento

Check (Verificar)

Verificacion automatizada (ESLint):

```
npm run lint # 0 errores, 0 advertencias
```

Casos de prueba manual:

#	Escenario	Entrada	Resultado Esperado	Resultado Obtenido
1	Transferencia exitosa	10u Paracetamol, Central->Miraflores	Stock origen -10, destino +10, COMPLETADO	✓
2	Stock insuficiente	999u producto con stock 4	ROLLBACK, ningun stock modificado, ERROR	✓
3	Producto inexistente	Codigo INVALIDO-999	ROLLBACK, ERROR con mensaje	✓
4	Polling en UI	Transferencia encolada	Spinner PENDIENTE -> Check COMPLETADO	✓

Resultados de verificacion:

- Atomicidad: 100% de las transferencias se completan con `COMMIT` o se revierten con `ROLLBACK`
- Trazabilidad: 100% de las transferencias quedan registradas en la tabla `transferencias`
- Tiempo de respuesta: El frontend recibe respuesta `202 Accepted` en < 100ms (sin bloqueo de red)

Act (Actuar)

Acciones correctivas identificadas:

1. Si se detectan transferencias con estado `PENDIENTE` que no se completan (stuck), implementar un trabajo de recuperacion (cron) que re-procese o cancele transferencias huérfanas después de un timeout
2. Si el volumen de transferencias crece significativamente, migrar la cola en memoria a una cola persistente tipo RabbitMQ o Redis Queue para mayor resiliencia
3. Si la frecuencia de errores de stock insuficiente es alta, implementar una validación previa en el frontend que consulte el stock actual antes de permitir la solicitud de transferencia
4. Documentar las lecciones aprendidas y actualizar el plan de calidad para el siguiente ciclo PDCA

Conclusión del Ciclo PDCA

La implementación del middleware de Message Queue siguiendo el ciclo PDCA garantiza que el proceso de transferencia de stock entre sucursales cumple con los estándares ISO 9000 de calidad, ofreciendo un sistema que es:

- **Confiable:** Transacciones ACID garantizan consistencia de datos
- **Trazable:** Cada transferencia es registrada con estado y mensaje
- **Responsivo:** El usuario no queda bloqueado durante el procesamiento
- **Mejorable:** El ciclo PDCA permite identificar y corregir desviaciones en iteraciones futuras

16. CLOUD COMPUTING — DESPLIEGUE

Plataforma Recomendada: Render + Supabase

El sistema está preparado para desplegarse en infraestructura cloud gratuita.

Requisitos de Despliegue

Componente	Servicio Cloud	Plan Gratuito
Backend (Node.js)	Render (Web Service)	750 hrs/mes
Base de Datos	Supabase (PostgreSQL)	500 MB, 2 proyectos
Frontend	Servido por Express en Render	Incluido
Cloud Storage	Supabase Storage o local	1 GB

Archivo de Configuración (render.yaml)

```
services:
  - type: web
    name: farmabol
    env: node
    buildCommand: npm install
    startCommand: npm start
    envVars:
      - key: PGHOST
        sync: false
      - key: PGPORT
        value: "5432"
      - key: PGUSER
        sync: false
      - key: PGPASSWORD
        sync: false
      - key: PGDATABASE
        sync: false
```

Variables de Entorno Requeridas

```
PORT=3000
PGHOST=db.supabase.co
PGPORT=5432
PGUSER=postgres
PGPASSWORD=tu_contrasena
PGDATABASE=postgres
```

Pasos de Despliegue

1. Base de Datos (Supabase):

- Registrarse en <https://supabase.com>
- Crear nuevo proyecto y obtener cadena de conexion
- La base de datos se crea automaticamente al iniciar el servidor por primera vez

2. Backend (Render):

- Subir repositorio a GitHub
- Crear nuevo Web Service en <https://render.com>
- Conectar repositorio
- Agregar variables de entorno (PGHOST, PGPORT, PGUSER, PGPASSWORD, PGDATABASE)
- Comando de inicio: `npm start`

3. Verificacion:

- Acceder a la URL publica generada por Render (ej: <https://mi-proyecto.onrender.com>)
- Iniciar sesion con admin/admin123
- Verificar que todas las funcionalidades operan correctamente

17. CLOUD STORAGE

Implementacion Actual

El sistema implementa almacenamiento de archivos (comprobantes de venta, fotos, codigos QR) utilizando:

- **Multer:** Middleware de Node.js para manejo de archivos multipart
- **Almacenamiento local:** Directorio `/uploads/` con nombre unico (timestamp + random)
- **Persistencia en BD:** Tabla `comprobantes` en PostgreSQL con metadatos

Endpoints de Cloud Storage

Metodo	Ruta	Descripcion
POST	/api/comprobantes/upload	Subir archivo (multipart/form-data)
GET	/api/comprobantes	Listar todos los comprobantes
GET	/api/comprobantes/venta/:ventaId	Comprobantes de una venta especifica

Ejemplo de Archivos Almacenados

```
Archivos en /uploads:
  1685987654321-143284756-factura001.pdf
  1685987654322-948372615-recibo.jpg
  1685987654323-123456789-qr-pago.png

Registros en BD (tabla comprobantes):
id | venta_id | nombre_archivo | url | tipo
1  | 1        | 16859...-factura001.pdf | /uploads/16859...-factura001.pdf | comprobante
2  | NULL     | 16859...-qr-pago.png | /uploads/16859...-qr-pago.png | qr
```

Migracion a Supabase Storage (Produccion)

Para entornos cloud, se recomienda reemplazar el almacenamiento local por Supabase Storage:

```
import { createClient } from '@supabase/supabase-js';
const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);

// Subir archivo
const { data } = await supabase.storage
  .from('farmabol-uploads')
  .upload(`comprobantes/${Date.now()}-${file.name}`, file);

// Obtener URL publica
const { data: { publicUrl } } = supabase.storage
  .from('farmabol-uploads')
  .getPublicUrl(data.path);
```

18. EVIDENCIA DE PRUEBAS FUNCIONALES

Funcionalidades Verificadas

#	Funcionalidad	Estado	Evidencia
1	Login ADMIN	✓	Inicio de sesion con admin/admin123
2	Login VENDEDOR	✓	Inicio de sesion con vendedor/venta123
3	Dashboard con KPIs	✓	Total ventas hoy, stock bajo, alertas vencimiento
4	CRUD Productos (ADMIN)	✓	Crear, listar, editar, eliminar productos
5	POS Registrar Venta	✓	Seleccionar productos, cobrar, descuento automatico de stock
6	Historial de Ventas	✓	Listado con detalle expandible por venta
7	Transferencia Asincrona	✓	Encolar, polling 2s, estado COMPLETADO/ERROR
8	Cloud Storage Upload	✓	Subir archivos, listar, filtrar por venta
9	Panel Arquitectura	✓	Documentacion tecnica integrada en la UI
10	Restablecer BD	✓	Reset a datos semilla

Capturas de Pantalla Recomendadas para el PDF

1. Pantalla de login con credenciales
2. Dashboard con KPIs y alertas de stock bajo

3. CRUD de productos (lista + formulario)
4. POS con carrito de compras
5. Confirmacion de venta con total
6. Historial de ventas con detalle expandido
7. Formulario de transferencia con estado PENDIENTE
8. Transferencia completada con check verde
9. Upload de comprobantes
10. Panel de arquitectura con metricas ESLint

19. ESTRUCTURA DEL REPOSITORIO

[illegible]

Scripts Disponibles

```
npm start      # Iniciar servidor en produccion
npm run dev    # Iniciar servidor en modo desarrollo (--watch)
npm run lint   # Ejecutar ESLint (analisis estatico)
```

Dependencias

Produccion: express, cors, dotenv, pg, multer **Desarrollo:** eslint, @eslint/js, globals

20. HISTORIAL DE COMMITS

Commits de Refactorizacion (etiquetados)

Commit	Tipo	Descripcion
ae3d2d2	ANTES	feat: implementar backend Express y conexion a PostgreSQL (codigo original sin transacciones)
6d7d52a	DESPUES	refactor: implementar transacciones SQL atomicas y validar stock en registrarVenta (codigo con ACID y FOR UPDATE)

Commits Adicionales de Refactorizacion

Commit	Descripcion
ed660ea	refactor: sanitizar mensajes de error en db.js para no exponer credenciales

Historial Completo (31 commits)

#	Commit	Descripcion
1	7610bc3	docs: documento completo del sistema FARMABOL con todos los diagramas SVGs referenciados
2	e97dc64	fix: reemplazar Unicode box-drawing y emojis en SVGs, usar dimensiones explicitas y fuentes seguras
3	56e0fb9	fix: corregir 9 inconsistencias entre diagramas UML y esquema BD real
4	09df8f0	fix: corregir lineas desconectadas, campos ER y actor en diagramas UML

5	99c9d16	fix: agregar declaracion XML UTF-8 a todos los SVG y convertir eslint-report a B&N
6	70c4771	docs: convertir todos los diagramas SVG a estilo UML clasico blanco y negro
7	ed660ea	refactor: sanitizar mensajes de error en db.js para no exponer credenciales
8	74a7a8f	fix: reemplazar icono Clipboard (inexistente) por Folder en UploadsScreen
9	428eef0	fix: corregir icono undefined en UploadsScreen y eliminar pestaña Arquitectura
10	52f5554	docs: agregar seccion de Cloud Storage a ARQUITECTURA.md
11	cad5f8a	docs: agregar configuracion de despliegue Render y reporte ESLint
12	7c3cbe5	feat: agregar pantalla de Cloud Storage con subida y descarga de archivos
13	3c515fe	feat: implementar cloud storage con tabla comprobantes y upload de archivos
14	409611d	docs: actualizar ARQUITECTURA.md con MQ, MTBF, disponibilidad y ciclo PDCA
15	5dcbfc6	docs: actualizar README con Message Queue, transferencias y nuevos endpoints
16	1395f93	docs: actualizar diagramas SVG para Hito 4 con Message Queue y Transferencia
17	6fdf5c0	feat: agregar TransferenciasScreen, pestana de transferencias y spinner
18	27db8e3	feat: agregar fecha_vencimiento y sucursal a formulario y alertas dashboard
19	c333b65	feat: agregar metodo transferirProducto y consultas derivadas al store
20	9a2f61e	feat: implementar MessageQueue y endpoints de transferencia asincrona
21	b4d4f2b	feat: implementar esquema multi-sucursal con fecha_vencimiento y tabla transferencias
22	72a27a6	fix: corregir comentarios XML ilegales en diagramas de clases y ERD
23	c4e0462	docs: agregar diagrama ERD en formato SVG
24	0f11630	docs: agregar diagramas UML en formato SVG y actualizar README.md
25	8f5275a	fix: agregar ruta raiz para servir FARMABOL.html
26	bac92f3	docs: generar reporte tecnico de arquitectura, base de datos y calidad estatica
27	4029f7a	feat: integrar frontend React con el backend y configurar calidad estatica
28	6d7d52a	refactor: implementar transacciones SQL atomicas y validar stock en registrarVenta (DESPUES)
29	ae3d2d2	feat: implementar backend Express y conexion a PostgreSQL (ANTES)
30	c15ce53	feat: prototipo de interfaz inicial para FARMABOL
31	0b6bc2d	Initial commit

Resumen de Commits de Refactorizacion

```
ANTES  (ae3d2d2):Codigo sin transacciones SQL ni bloqueo de filas
              -> Riesgo de inconsistencias y condiciones de carrera

DESPUES (6d7d52a):Codigo con transacciones ACID (BEGIN/COMMIT/ROLLBACK)
              -> Bloqueo FOR UPDATE, atomicidad, manejo seguro de errores
```

DECLARACION JURADA

"El codigo es de mi autoria, no use IA para generar las respuestas ni el codigo del sistema."

*Documento generado como parte de los entregables de los Hitos 3 y 4 de la materia Software II.
Sistema: FARMABOL — Control de Inventarios y Ventas Fecha: Junio 2026*